

METODOLOGIE AGILI E EXTREME PROGRAMMING

Michele Marchesi

marchesi@unica.it

**Corso di Ingegneria del Software
A.A. 2025/26**



...un piccolo passo indietro:

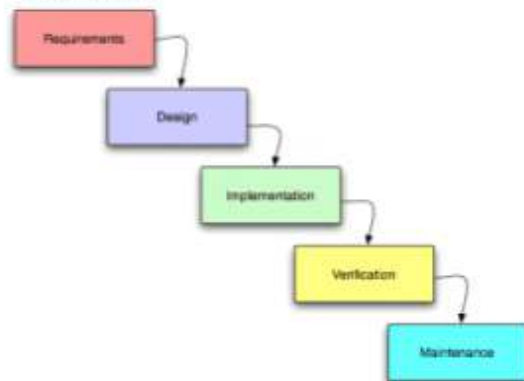
Metodi di sviluppo plan-driven

- ◆ Approccio classico dell'ingegneria del software, fondato su processi ben definiti
 - passi standard: requisiti → progetto → realizzazione
- ◆ Enfasi su:
 - Contratto con il cliente
 - Pianificazione
 - Seguire un piano
 - Processo di sviluppo (e tool)
 - Documentazione (completa)
 - Ogni fase termina con una milestone
 - ◆ Produzione artefatti

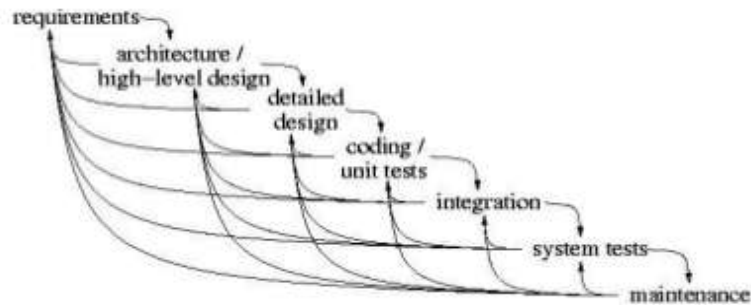


Metodi di sviluppo plan-driven e loro correttivi

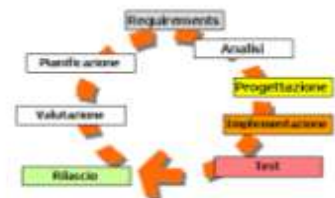
Cascata



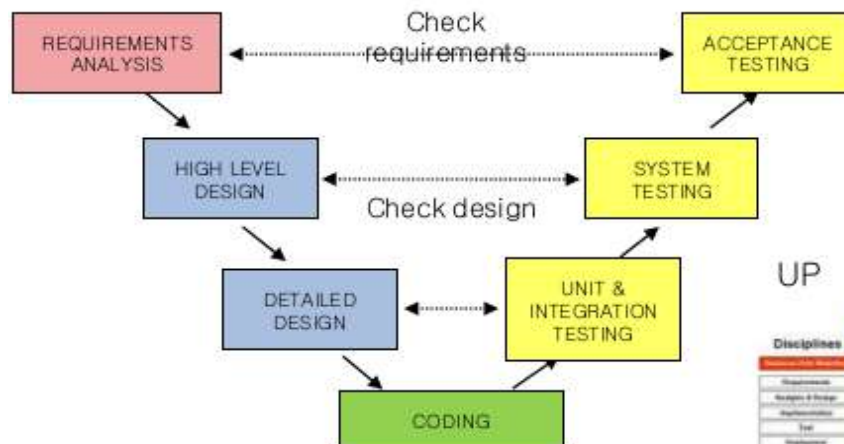
Cascata con iterazioni e feedback



Modelli iterativi



V-model



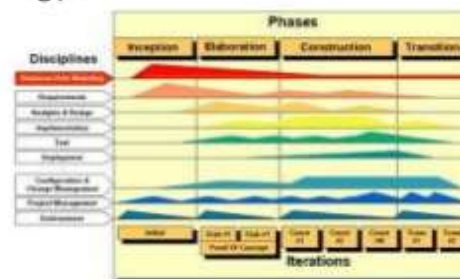
Modelli incrementali



Modello a spirale



UP



Critiche ai metodi plan-driven

L'economia Internet chiede **flessibilità e velocità**

- ◆ Bisogna rispondere ai cambiamenti in modo veloce e lunghi cicli di sviluppo e tanta pianificazione non aiutano ...
- ◆ È molto difficile capire i bisogni del cliente, spesso poco chiari anche a lui, e “formalizzarli” nei requisiti
- ◆ Viene messa in crisi:
Qualità del processo → Qualità del prodotto
- ◆ Programmare è un arte: non un processo industriale
- ◆ Software = prodotto particolare
 - Elemento logico, immateriale, non “fisico”
 - Estremamente “malleabile”
 - Tutti i costi sono nell'ingegnerizzazione, non nella fabbricazione



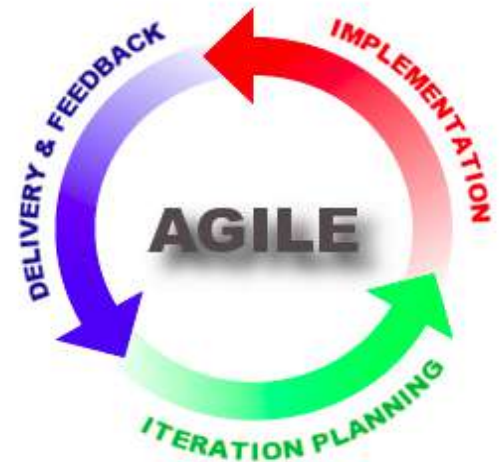
Clienti stupefatti dei continui ritardi

Clienti insoddisfatti della bassa qualità del software



Cosa sono le Metodologie Agili (AM)

- ◆ Non sono “un ritorno alla codifica disordinata e non documentata” (*Cowboy Coding*) ma **processi ben definiti** che mantengono il controllo sulla qualità del software
- ◆ Un **processo agile** è un processo adattativo ed incrementale
 - continua verifica di rispondenza ai requisiti
 - si sviluppa per livelli crescenti di complessità
 - in grado di gestire l'imprevedibilità



Caratteristiche delle Metodologie Agili (AM)

- ◆ Requisiti soprattutto verbali
- ◆ Analisi e progetto fatte informalmente minimizzando l'uso di diagrammi formali
- ◆ Minima documentazione oltre al codice
- ◆ Il processo è semplice, ma deve essere seguito con molta disciplina
- ◆ I membri del gruppo condividono dei valori
- ◆ Molto produttive per piccoli gruppi co-locati

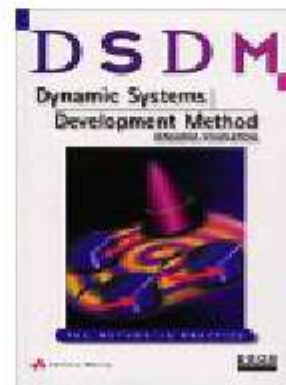
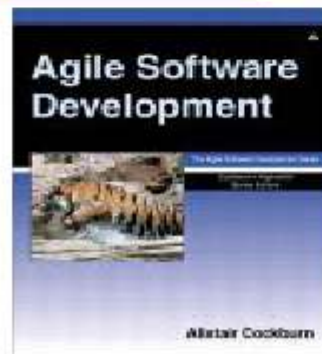
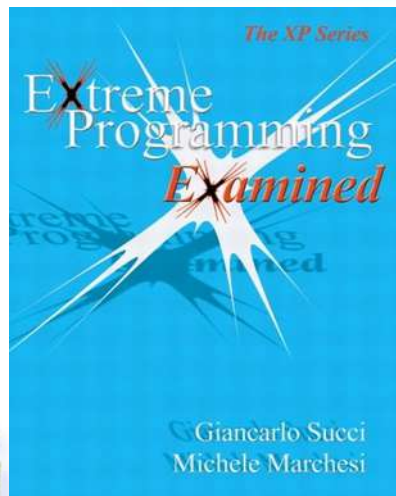
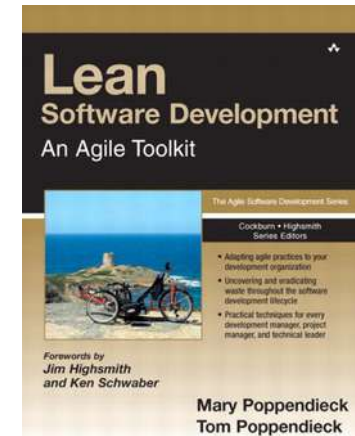
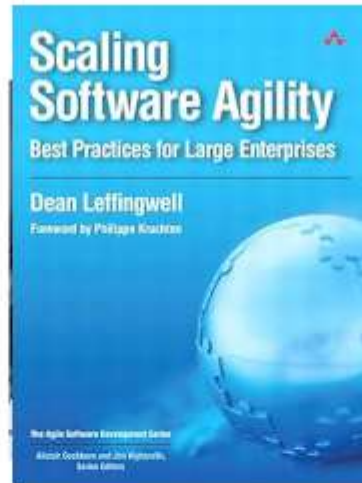


Agile Methodologies

- ◆ **SCRUM** (introdotta nel 1990), di Ken Schwaber & Jeff Sutherland
- ◆ **Extreme Programming (XP)** di Kent Beck
- ◆ **Crystal Methods** di Alistair Cockburn
- ◆ **Lean Software Development** di Howell e Poppendieck
- ◆ **Feature Driven Development (FDD)** di Jeff De Luca & Peter Coad
- ◆ **DSDM** di Arie van Bennekum & Jennifer Stapleton
- ◆ **SAFe (Scaled Agile Framework)** di Dean Leffingwell, per organizzazioni più grandi



Agile methodologies



Manifesto for Agile Software Development (2001)

- ◆ Stiamo scoprendo modi migliori di creare software, sviluppandolo e aiutando gli altri a fare lo stesso
- ◆ Grazie a questa attività siamo arrivati a considerare importanti:

Gli individui e le interazioni

più che

i processi e gli strumenti

Il software funzionante

più che

la documentazione esaustiva

La collaborazione col cliente

più che

la negoziazione dei contratti

Rispondere al cambiamento

più che

seguire un piano

- ◆ Fermo restando il valore delle voci a destra, consideriamo più importanti le voci a sinistra.

Principi dell'Agile Manifesto

1. La nostra massima priorità è soddisfare il cliente rilasciando software di valore, fin da subito e in maniera continua.
2. Accogliamo i cambiamenti nei requisiti, anche a stadi avanzati dello sviluppo. I processi agili sfruttano il cambiamento a favore del vantaggio competitivo del cliente.
3. Consegniamo frequentemente software funzionante, con cadenza variabile da un paio di settimane a un paio di mesi, preferendo i periodi brevi.
4. Committenti e sviluppatori devono lavorare insieme quotidianamente per tutta la durata del progetto.



Principi dell'Agile Manifesto

5. Fondiamo i progetti su individui motivati. Diamo loro l'ambiente e il supporto di cui hanno bisogno e confidiamo nella loro capacità di portare il lavoro a termine.
6. Una conversazione faccia a faccia è il modo più efficiente e più efficace per comunicare con il team ed all'interno del team.
7. Il software funzionante è il principale metro di misura di progresso.
8. I processi agili promuovono uno sviluppo sostenibile. Gli sponsor, gli sviluppatori e gli utenti dovrebbero essere in grado di mantenere indefinitamente un ritmo costante.



Principi dell'Agile Manifesto

- 9. La continua attenzione all'eccellenza tecnica e alla buona progettazione esaltano l'agilità.
- 10. La semplicità - l'arte di massimizzare la quantità di lavoro non svolto - è essenziale.
- 11. Le architetture, i requisiti e la progettazione migliori emergono da team che si auto-organizzano.
- 12. A intervalli regolari il team riflette su come diventare più efficace, dopodiché regola e adatta il proprio comportamento di conseguenza.



XP - eXtreme Programming

- ◆ XP: metodica di sviluppo software
- ◆ Introdotta nel '97 da Kent Beck nel progetto C3 in Daimler Chrysler
- ◆ XP ribalta alcune assunzioni fondamentali dell'ingegneria del software
- ◆ XP aumenta sia la qualità che la produttività
- ◆ Versione 2 nel 2004 (New XP)

Kent Beck a XP2001,
Villasimius



XP in sette frasi

1. Piccoli gruppi (max. 10-15), programmazione a coppie
2. Testing prima della codifica
3. Integrazione quotidiana, rilasci continui
4. Guidata da brevi casi d'uso (*storie*), con tecniche precise di stima dei tempi e costi
5. I programmi sono ristrutturati continuamente
6. La documentazione è nel codice (intention revealing code)
7. Scrivi il più semplice sistema che possa funzionare!



Storia e successi

- ◆ Nel progetto C3, 12-15 programmatori hanno scritto in 2 anni un sistema paghe e stipendi del gruppo Chrysler!
 - *Ci sono voluti 4 anni e 30 programmatori per capire che lo sviluppo precedente era fallito...*
- ◆ Il libro di Beck sull' XP ha venduto oltre 200.000 copie e ha destato l'interesse per le MA
- ◆ Ora l'XP come tale è molto poco usata, ma quasi tutte le pratiche XP sono applicate ovunque



Motivazioni dell'XP

- ◆ E' basata su un insieme di tecniche ciascuna già usata e trovata efficace
- ◆ L'XP adotta tutte tali tecniche al massimo grado, di qui il termine **estrema**
- ◆ Il più possibile “leggera” e flessibile

XP is a lightweight methodology for small to medium sized teams developing software in the face of vague or rapidly changing requirements.

Kent Beck



I quattro valori dell'XP

- ◆ Come tutte le metodologie agili, l'XP si fonda su dei **valori** condivisi dal gruppo
- ◆ Tali valori sono il riferimento per tutto il processo
- ◆ I valori dell'XP sono quattro:
 - **Comunicazione**
 - **Semplicità**
 - **Feedback**
 - **Coraggio** → + **Rispetto** (New XP)



Comunicazione

- La maggior parte dei problemi e degli errori provengono da difetti di comunicazione
- Le comunicazioni tra sviluppatori, e tra questi e i clienti devono essere massimizzate ad ogni costo
- Il progetto e il codice devono essere comprensibili e aggiornati
- Tra tutte le forme di comunicazione, la più efficace è quella diretta interpersonale



Semplicità

- Fa la cosa più semplice che possa funzionare
- La semplicità favorisce la comunicazione, riduce il lavoro e permette una qualità migliore
- Non si deve pianificare per futuri riusi possibili (ma non certi)
- Nuove caratteristiche potranno essere facilmente aggiunte, quando richieste



Feedback

- ◆ Si deve sempre poter misurare il sistema e sapere di quanto esso si sta discostando dalle caratteristiche volute.
- ◆ Gli strumenti fondamentali:
 - Contatto continuo col cliente
 - Disponibilità di un insieme di test automatici, sviluppati insieme al sistema stesso

Coraggio

- **Tutte le metodologie ed i processi sono strumenti per affrontare e ridurre le nostre paure**
- Quanto maggiori sono le paure, tanto più grandi e pesanti le metodologie
- Comunicazione, semplicità e “suite” di test automatici permettono di affrontare con grande coraggio anche grandi mutamenti dei requisiti e ristrutturazioni sostanziali del software
- Tutte le volte che si intravede la possibilità di un miglioramento, occorre intervenire sul software senza paura (*refactoring* continuo)



Rispetto

- Se ai membri di un gruppo non importa dei compagni e del lavoro fatto, l'XP non può funzionare
- Se ad essi non importa il progetto, nulla può salvarlo
- Occorre rispetto per i propri colleghi e per il contributo che ciascuno dà, per la propria organizzazione, per le persone che saranno coinvolte dal sistema che si sta scrivendo



Attività dell'XP

◆ **Pianificazione**

- Ricavare il maggior feedback possibile da clienti, compagni del gruppo e dal sistema stesso
- Pianificare in modo realistico e tenendo conto del feedback il proprio lavoro

◆ **Progetto**

- Sviluppare una struttura concettuale del sistema coerente e semplice che evolve in continuazione



Attività dell'XP

◆ **Codifica**

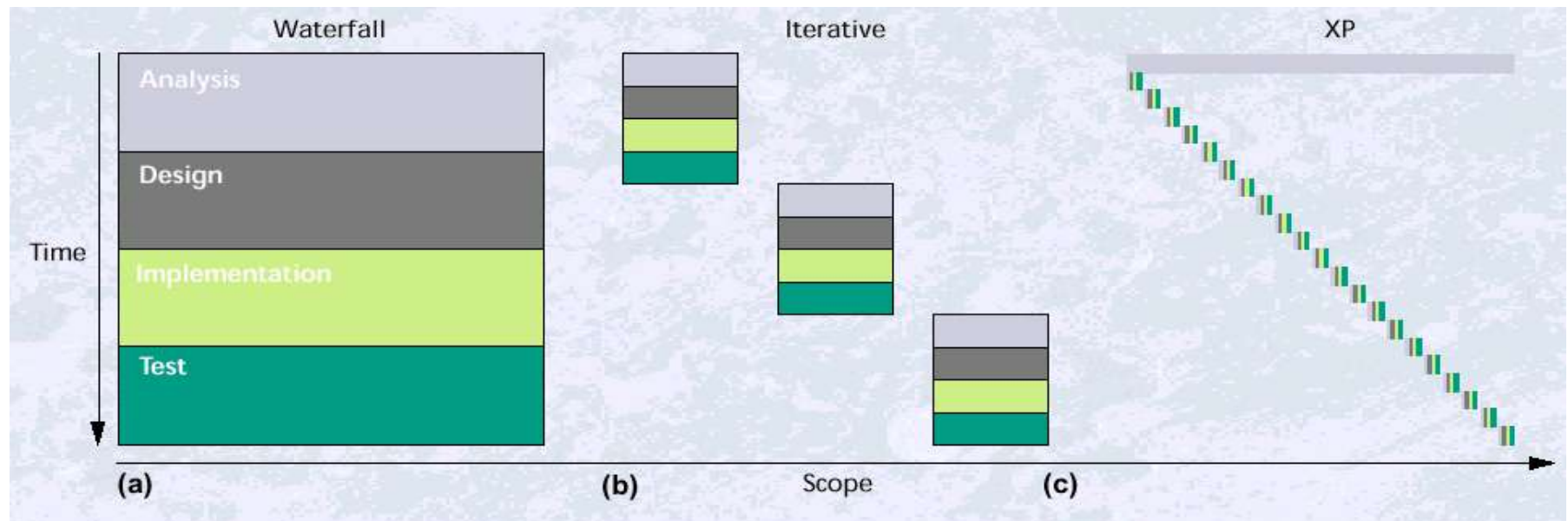
- Le pratiche per produrre il codice, integrarlo e rilasciarlo
- E' ciò che alla fine produce il valore

◆ **Testing**

- Come collaudare il sistema e come risolvere i problemi trovati
- Deve essere il più possibile automatizzato



Processi di produzione del software



Le pratiche dell'XP

- L'XP si basa su dodici pratiche fondamentali (più altre introdotte successivamente)
- Esamineremo tali pratiche suddividendole per attività, anche se alcune non sono perfettamente inquadrabili
- Le pratiche funzionano ***in modo XP*** se applicate tutte e al massimo
- Si possono comunque trarre benefici anche applicandole singolarmente
- Almeno 8 su 12 sono oggi usate da (quasi) tutti i team agili



Le 12 pratiche dell'XP

1. **Planning Game:** User Stories, Project Velocity (*deriva da Scrum*)
2. **Rilasci brevi:** iterazioni di 2 settimane, rilasci di 2-3 mesi
3. **Metafora:** una visione comune del problema tra cliente e team
4. **Semplicità di progetto:** fa la cosa più semplice che possa funzionare

Codici dei colori:

Usatissima

Poco usata

Non più usata



Le 12 pratiche dell'XP

5. *Testing Continuo*: scrivere i test prima del codice – tutti i test sono automatici

6. *Refactoring*: migliora e semplifica il codice quando necessario, senza paura!

7. *Pair Programming*: si programma sempre in coppia

8. *Integrazione Continua*: si integra almeno una volta al giorno



Le 12 pratiche dell'XP

- 9. *Proprietà collettiva del codice*:** tutti possono modificare il codice
- 10. *Cliente sul posto*:** per fornire i requisiti e dare feedback subito
- 11. *Ritmo sostenibile (anche: settimana di 40 ore)*:** niente straordinari, team riposato e concentrato
- 12. *Standard di codifica*:** condivisi e uguali per tutto il team



Pianificazione

- *Planning Game* per pianificare e interagire col cliente -> deriva da Scrum, introdotto nel 1990 e che precede XP (introdotto nel 1997)
- *Storie (User stories)* per raccogliere i requisiti (**già viste**)
- *Punti storia* per controllare il processo
- *Iterazioni* di 1 - 4 settimane
- *Riunione di Pianificazione dell'Iterazione* all'inizio di ogni iterazione
- *Riunione in piedi (Stand-up Meeting)* ogni giorno



US e processo agile (XP e Scrum)

- 1 Il cliente (o il PO) scrive le user stories su index card, su un foglio elettronico o su un *tool CASE*
 - 2 Il cliente (o il PO) definisce i test di accettazione per le storie
 - 3 Il team di sviluppo stima le storie
 - 4 Il cliente o il PO sceglie le storie per il rilascio
- XP e Scrum sono processi incrementali e iterativi, con **rilasci frequenti**
 - Un rilascio avviene di solito ogni 3 mesi
 - Per ogni rilascio, si effettuano iterazioni (o *sprint*) di 1-2 settimane



Processo agile: release (rilascio)

- Ad ogni **release** (rilascio) viene rilasciata una versione funzionante del sistema, che implementa un certo numero di User Stories
- **All'inizio di ogni rilascio** si effettua una riunione in cui il cliente sceglie le storie da implementare
 - Si rivede brevemente lo stato di tutto il progetto
 - Il cliente (o PO) può modificare, aggiungere o eliminare storie.
 - La modifica e l'eliminazione di storie già implementate sono considerate **nuove storie**
 - Gli sviluppatori possono stimare nuovamente le storie, in base ai risultati del rilascio precedente



Processo agile: iterazioni

- Nelle iterazioni (in Scrum: *sprint*) viene pianificato lo sviluppo per la nuova iterazione. Ogni ciclo inizia con una riunione di pianificazione
- Durante la **riunione di pianificazione dell'iterazione**:
 - Il cliente o il PO sceglie le storie da implementare in quella iterazione
 - Il cliente può aggiungere/modificare/eliminare storie.
 - La modifica e l'eliminazione di storie già implementate sono considerate nuove storie
 - Gli sviluppatori suddividono ciascuna storia in parti più semplici, noti come **Task**
 - Gli sviluppatori stimano i task e i TA



Processo agile: iterazione

- Dopo la divisione delle US in task, ogni sviluppatore **sottoscrive** dei task e/o dei TA
- Durante l'iterazione (o Sprint), lo sviluppatore implementa i task (eventualmente con un compagno: **Pair Programming**)
- A ogni task implementato, si aggiorna la *Burndown Chart* dello Sprint
- Alla fine dell'iterazione il cliente, o il Product Owner, **accetta** le storie completate
- Le iterazioni sono di lunghezza fissa: non si possono prolungare (**time-boxed**)!





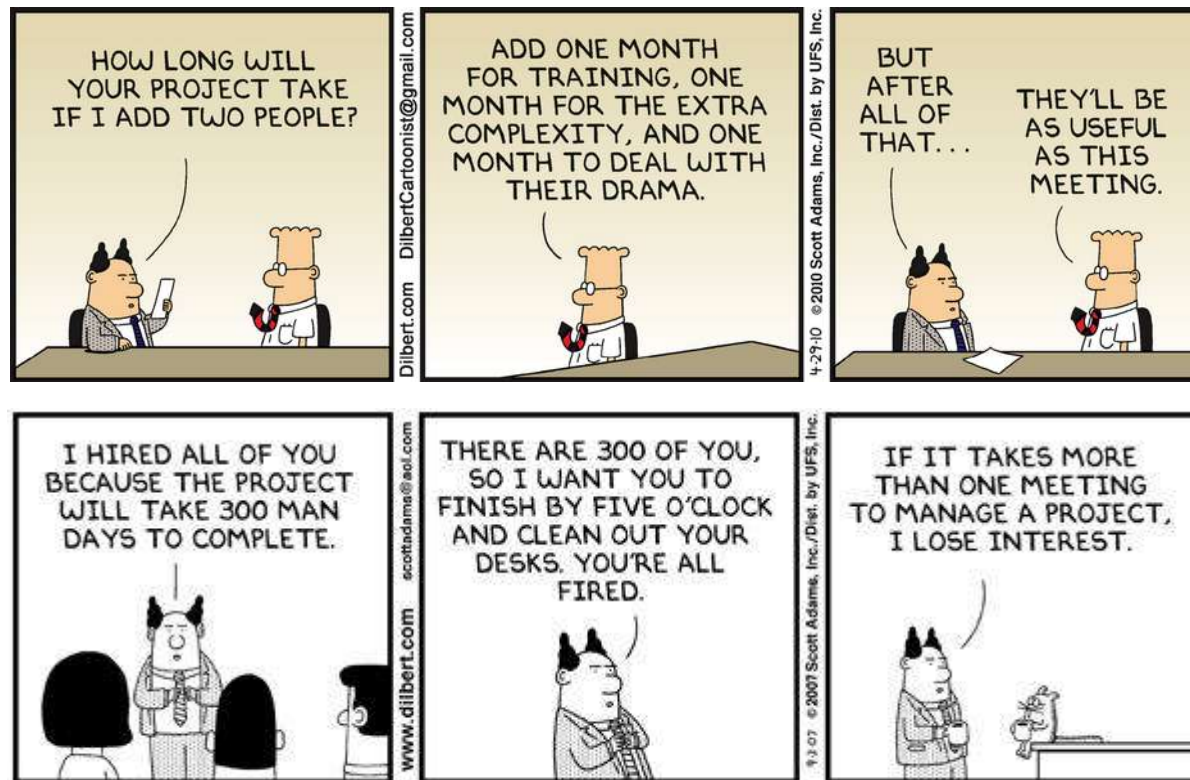
Planning Game

- Il progetto è completamente guidato dal valore fornito al cliente (*Business Value*)
- Il Business Value appartiene al cliente!
- ***Quattro variabili:***
 - Funzionalità
 - Tempo
 - Risorse (molto nonlineare)
 - Qualità: sempre al massimo (ragionevole)



Le risorse sono non lineari!

- *Aggiungere risorse a un progetto in ritardo, farà aumentare il ritardo (Fred Brooks)*



© Scott Adams, Inc./Dist. by UFS, Inc.

Il contratto tra cliente e sviluppatori

- Il cliente (o PO) può fissare 3 delle 4 variabili (ma **qualità e risorse sono già fisse**)
- Gli sviluppatori fissano la quarta
- Di solito, il cliente sceglie le storie (funzionalità) da implementare e comunica il tempo limite
- Gli sviluppatori dicono quante storie possono implementare nel tempo dato
- Il cliente sceglie quali storie realizzare effettivamente
- ***Il cliente non può fissare tutte e 4 le variabili!!***



Il processo del Planning Game

- 1) Il cliente* scrive le storie su delle schede
- 2) Il cliente definisce i test di accettazione per le storie
- 3) Il programmatore stima le storie
- 4) Il cliente sceglie le storie per il rilascio
- 5) Il cliente sceglie le storie per l'iterazione
- 6) Il programmatore definisce i *task* per ogni storia
- 7) Il programmatore accetta e stima i task
- 8) Il programmatore sviluppa i task (con un compagno)
- 9) Il cliente accetta le storie completate

 *(in Scrum, cliente = Product Owner)

Story points

- Ogni storia ha una stima in “punti-storia”
- Dipendono dalla dimensione e dalla complessità del lavoro da fare
- Adimensionali ma **lineari**:
 - una US da 10 punti dovrebbe essere realizzata con un lavoro doppio rispetto a una US da 5 punti
- Per iniziare, potrebbero essere la stima in giorni-uomo o ore-uomo necessarie per completare la storia



La stima in punti storia

- Forza l'uso di **stime relative**
 - Esistono studi che dimostrano che le persone stimano meglio in senso relativo
 - È difficile stimare l'altezza di un edificio posto a 300 m. di distanza, ma è facile stimare che un edificio è alto il doppio di un altro lì vicino!
- Si focalizzano sulla stima della **dimensione**, non della durata
 - La durata si deriva empiricamente vedendo in quanto si completa un'iterazione
- Si fanno stime in **unità additive**



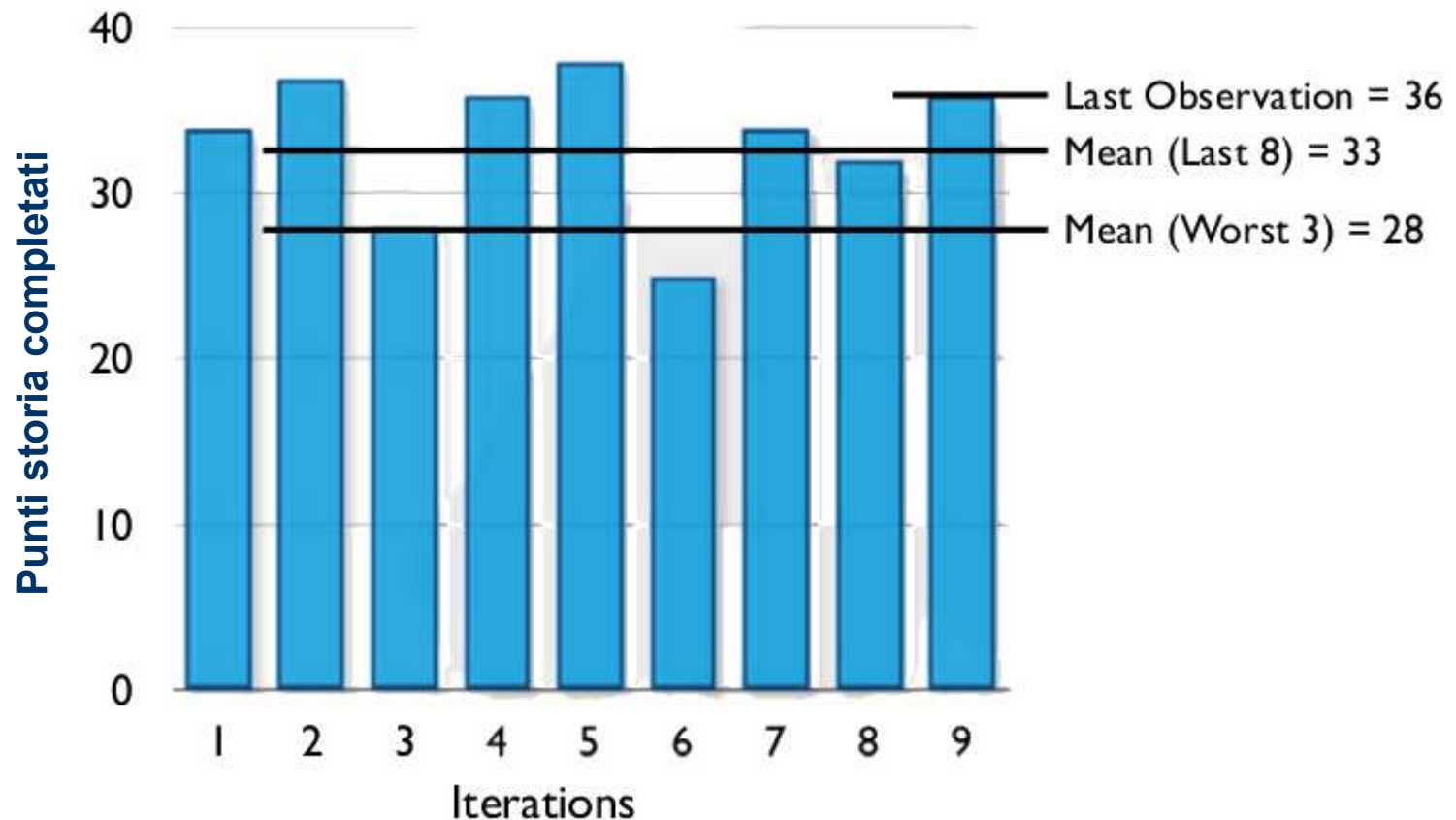
Stima della velocità e iterazioni

- Durante la riunione di pianificazione dell'iterazione, si stima la *Project Velocity* o *velocità di progetto* (numero di Story Points che il team può implementare in un'iterazione)
- Si assegnano a ogni iterazione le US di maggior priorità – *le decide il cliente!*
- Le iterazioni sono ***time-boxed***
- Se non si riesce a completare qualche US in un'iterazione, questa si sposta alla successiva
- Si aggiorna di conseguenza la velocità di progetto

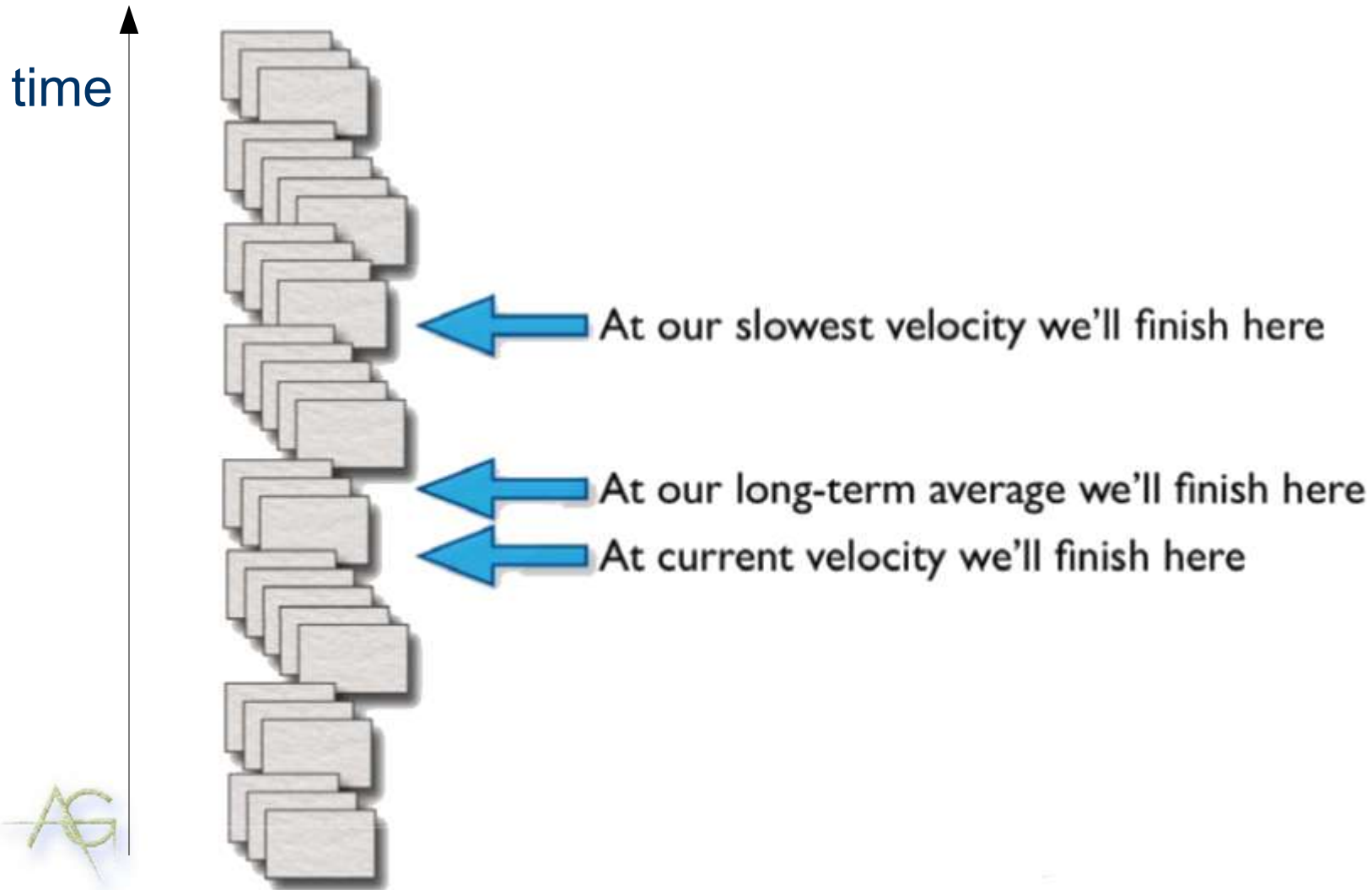


Stima della Project Velocity

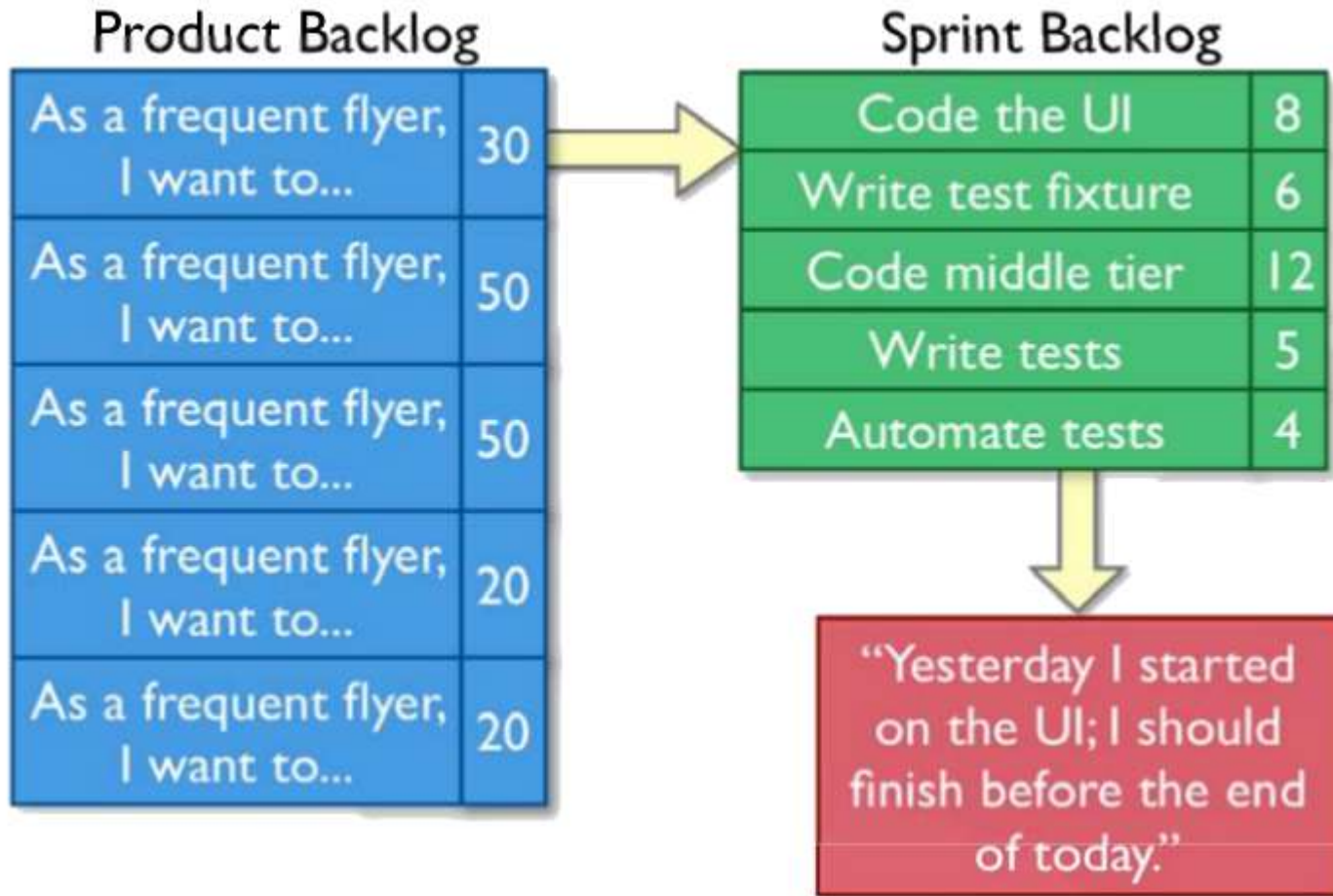
- Possibili vari modi per farlo



Estrapolare con la velocità



Una storia e i suoi task



Burndown Charts

- Ogni giorno, si registra il numero di Story Points, o di ore di lavoro, **che rimangono da fare**
- A livello di progetto, il *Burndown di progetto* è il numero di Story Points delle US ancora da implementare
- A livello di iterazione, il *Burndown di Sprint* è il numero di ore di lavoro che restano per finire i Task dell'iterazione (Sprint)
- I Burndown danno al team la sensazione dei progressi del lavoro
- E' un tipico strumento di Scrum



Andamento delle Burndown Charts

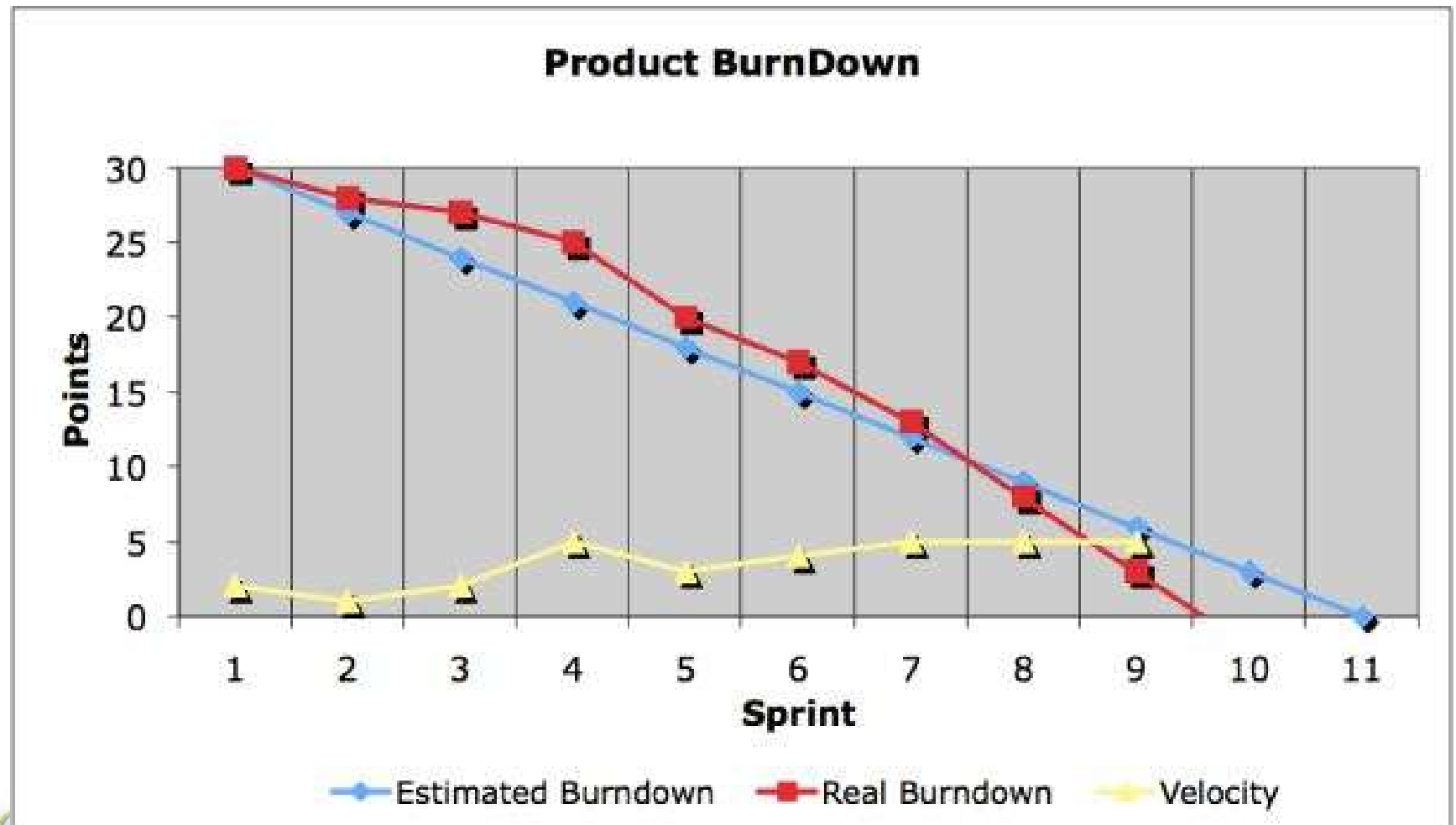
- Il numero di punti storia o di ore che mancano prima della fine del progetto o dell'iterazione potrebbe anche **aumentare**, a causa di:
 - Nel burndown di progetto, aggiunta di storie extra da implementare (dopo una riunione di pianificazione)
 - Nel burndown di iterazione, cambiamento in aumento della stima di qualche task, che si è rivelato più difficile del previsto
- Ovviamente, tali numeri possono anche essere rivisti al ribasso, se il progetto procede meglio del previsto



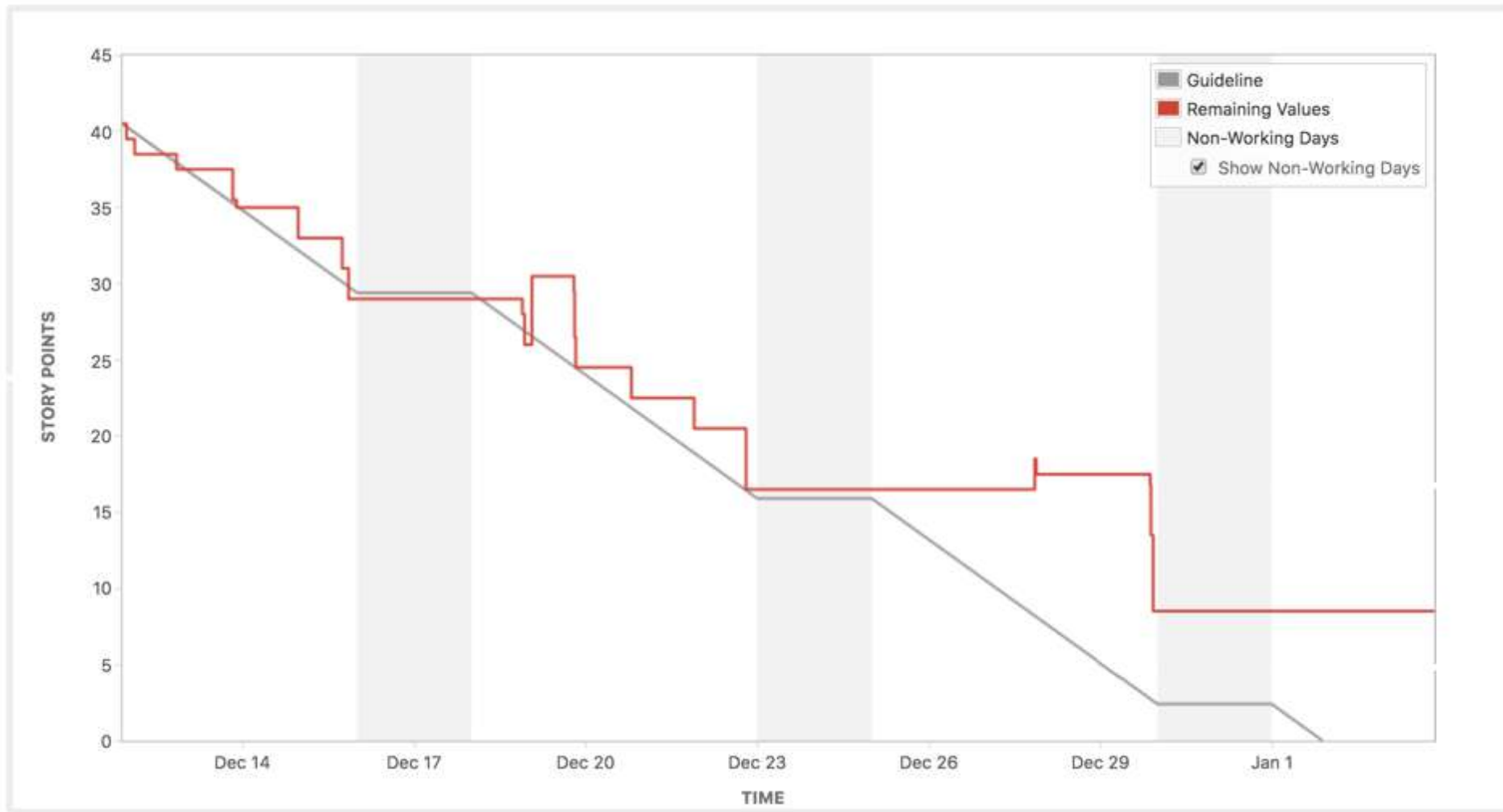
Una burndown chart di iterazione



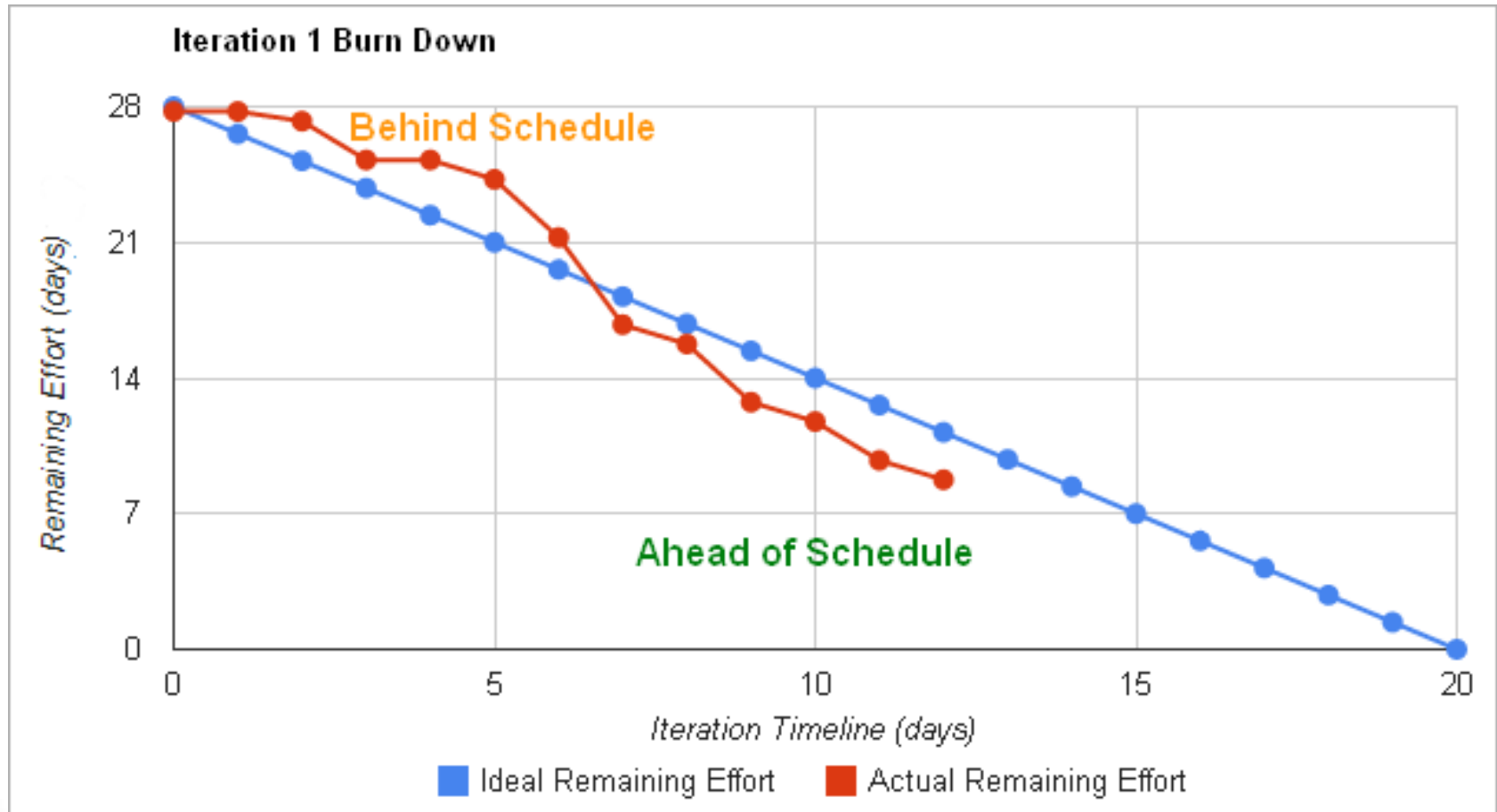
Una b.c. di Release



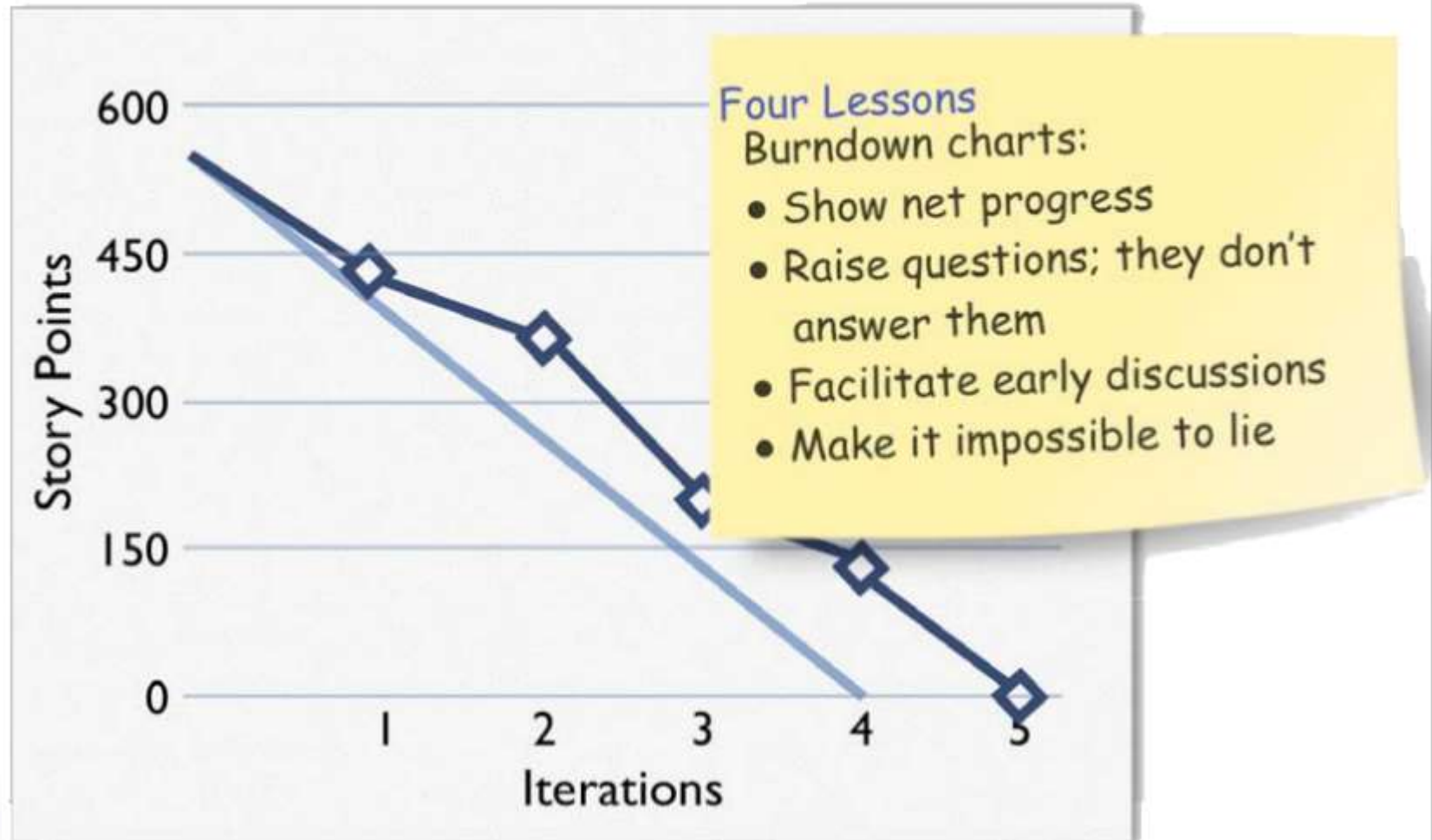
Un'altra b.c. di iterazione da Jira



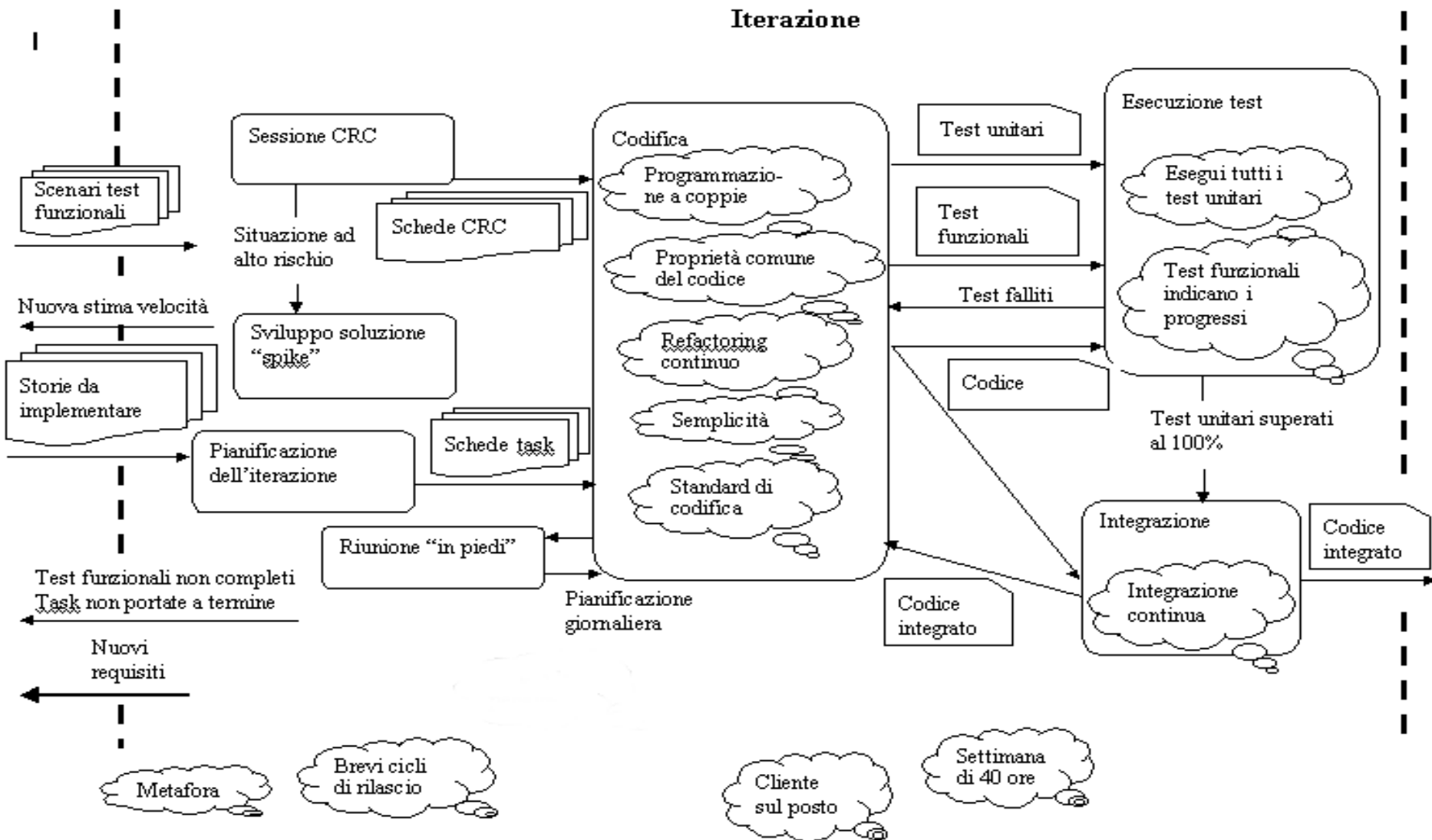
Un'altra b.c. di iterazione da un mese



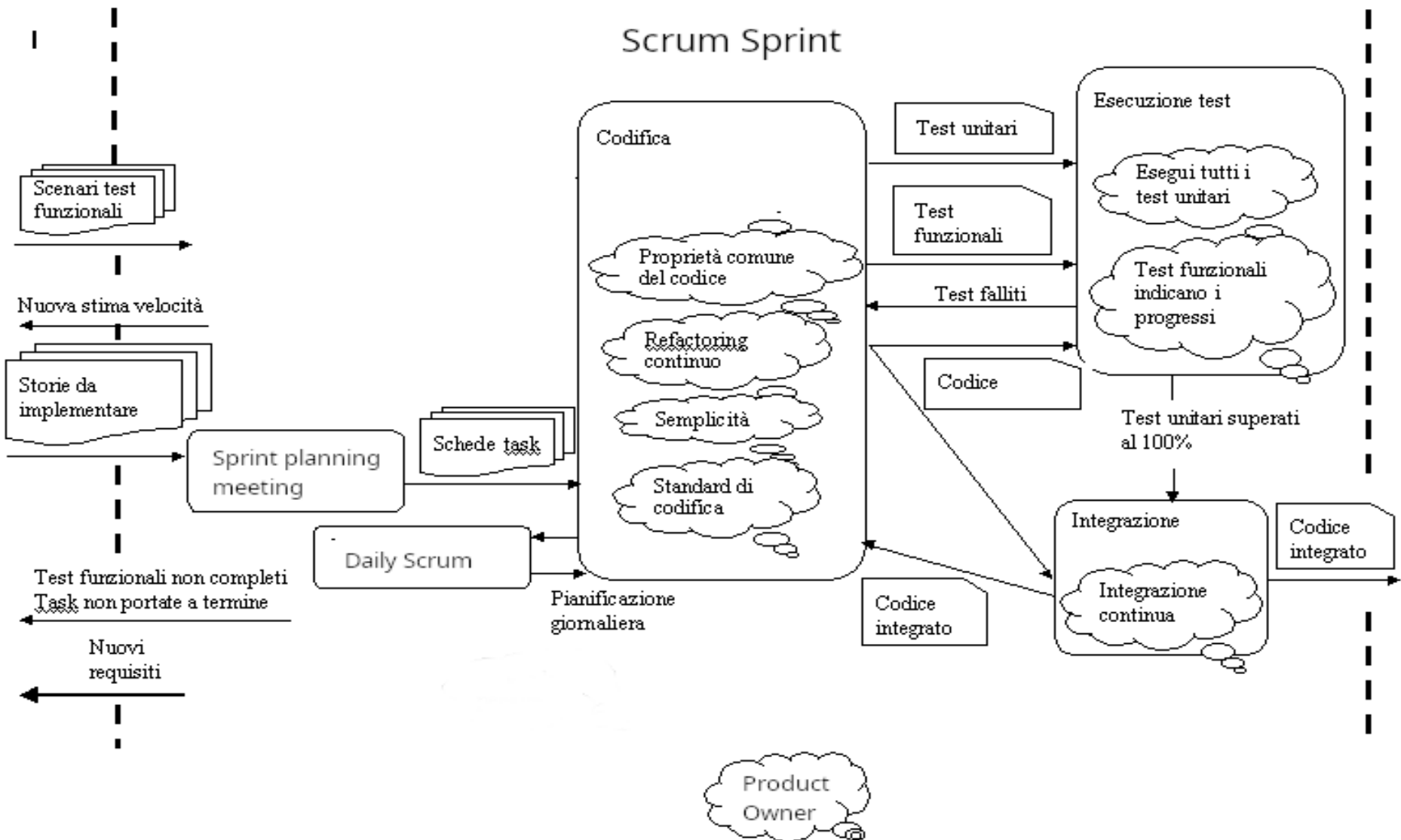
Una burndown chart di release



Iterazione e pratiche XP



Iterazione Scrum (con pratiche tipiche)



Controllo dell'iterazione

- ◆ Una *riunione in piedi* al giorno per fare il punto e evidenziare i problemi
- ◆ Se l'iterazione è in ritardo, il cliente sceglie che storie differire alla successiva
- ◆ Un ritardo/anticipo fa variare la stima della velocità del gruppo
- ◆ Ogni 2-3 iterazioni è bene quindi rivedere la pianificazione del rilascio



Progettazione

- ◆ **Semplicità:** *fa la cosa più semplice che possa funzionare*
- ◆ *YAGNI (You aren't gonna need it!)*
- ◆ Scegli una **Metafora del Sistema**
- ◆ Usa le *Schede CRC* per le sessioni di progetto
- ◆ Crea *Soluzioni “Spike”* per ridurre il rischio
- ◆ *Ristruttura (Refactor)* ovunque e tutte le volte che è possibile



Fa la cosa più semplice che possa funzionare

- Un sistema semplice è più facile da comprendere e da mantenere
- La semplicità non è facile da ottenere
- Ad essa si tende gradualmente, col refactoring
- Non progettare mai un sistema pensando a possibili futuri riusi:
 - Si aggiungono costi e complessità oggi
 - Non c'è la garanzia che ciò sia veramente utile domani
 - Se il sistema è semplice, si potranno facilmente aggiungere ulteriori caratteristiche quando necessario



YAGNI: You Aren't Gonna Need It!

- ***Non ti serve! -- Non ne hai bisogno!***
- Quando si è in dubbio se aggiungere al sistema qualcosa, chiedetevi se vi serve realmente **ora**
- Se la risposta è no, non complicate il sistema con qualcosa non necessario
- Il modo migliore per procedere velocemente è scrivere il meno codice possibile!
- Il miglior modo per avere meno errori è avere meno codice!



Schede CRC

- ◆ Come passare dalle storie (descrizione funzionale) alla struttura ad oggetti del sistema?
- ◆ Il metodo suggerito dall'XP è l'analisi con le schede CRC (Classe, Responsabilità, Collaborazione)
- ◆ E' una tecnica di OOA molto usata e collaudata, introdotta da Beck e Cunningham nel 1989



Soluzioni Spike

- Una “*spike solution*” è un programmino scritto per sperimentare una soluzione
- E' un prototipo, quasi sempre “usa e getta”
- Lo scopo principale è la riduzione del rischio

Quando c'è un problema che potrebbe mettere a rischio il progetto, dedicate una coppia di programmatori a sviluppare una soluzione spike per una o due settimane, per esplorare la situazione e trovare possibili soluzioni



Codifica

- Tutto il codice è *programmato a coppie*
- *Il codice è di tutti (Collective Code Ownership)*
- Integrate spesso (una volta al giorno).
- Rilasciate spesso (una volta ogni 1-3 settimane).
- Ottimizzate solo all'ultimo
- *Settimana di 40 ore (niente straordinari!)*
- Il codice deve seguire precisi standard: *la documentazione è nel codice*



Programmazione a coppie (Pair Programming)

- Tutto il codice deve essere scritto da una coppia di programmatori
- La coppia si alterna alla tastiera
- Il più inesperto è bene stia alla tastiera per più tempo
- Le coppie cambiano in continuazione
- Le coppie si formano quando il programmatore responsabile di un task invita un altro libero a partecipare alla codifica (***non si può dire no!***)



Programmazione a coppie (Pair Programming)

- ◆ I due ruoli della coppia
 - **Il programmatore A:** alla tastiera pensa al modo migliore per implementare questo metodo
 - **Il programmatore B:** pensa in modo più strategico:
 - Questo approccio funzionerà?
 - Ci sono altri test unitari da aggiungere?
 - C'è modo di semplificare il sistema?



Programmazione a coppie



Vantaggi della programmazione a coppie

- ◆ Il codice prodotto è di qualità superiore
- ◆ Ci sono sempre almeno due persone che conoscono a fondo ogni parte del sistema
- ◆ L'inserimento e l'addestramento di nuovi membri del gruppo è molto facilitato.
- ◆ I due programmatori tendono ad essere molto più concentrati
- ◆ Esistono studi quantitativi che dimostrano che la programmazione a coppie è più conveniente di quella singola



Proprietà collettiva del codice

- Nell'XP, il sistema è accessibile a tutti
- Chiunque può cambiare qualsiasi parte del codice, ovunque nel sistema, in ogni momento
- Ciò semplifica la gestione del progetto
- L'approccio può funzionare perché:
 - Il sistema è semplice e leggibile
 - C'è una comunicazione diretta tra i programmatori
 - Si ha a disposizione una “suite” completa di test automatici



Integrazione continua

- Il sistema deve essere integrato una volta al giorno
- Aspettare più di due giorni è indice di problemi
- Si usa una macchina per l'integrazione
- Man mano che le coppie terminano un task, o hanno prodotto del codice integrabile, vanno alla macchina per l'integrazione e installano il nuovo codice
- Poi eseguono i test automatici e risolvono i problemi
- Se tutto è OK, la nuova versione “ufficiale” del sistema diventa quella con la modifica
- Pratica oggi grandemente facilitata dai sistemi di configuration management



Niente straordinari

- Per produrre software di qualità occorre essere freschi e riposati
- Non tutti sopportano gli stessi ritmi di lavoro, ma tutti hanno bisogno di riposo e di tempo per se stessi
- La necessità di effettuare straordinari è indice di problemi nel processo di sviluppo
- Gli straordinari sono ammessi solo per non più di una settimana, ogni tanto



Standard di codifica

- L'XP non prescrive documentazione formale, oltre al codice stesso
- Il codice deve essere molto leggibile e ben strutturato
- La prima condizione è che sia semplice e ben progettato
- La seconda condizione è che sia scritto con le stesse regole da tutti i membri del gruppo:
 - Come dare i nomi alle varie entità del programma
 - Standard di formattazione del codice per ottenere la massima leggibilità.



Codice “Terso”

- Il miglior codice è conciso ma leggibile: è *terso*
- Il codice terso è difficile da ottenere, richiede:
 - Semplicità di progetto
 - Convenzioni di nomi seguite sempre
 - Struttura gerarchica ai vari livelli di astrazione
- Si chiama anche “***Intention revealing code***”
- La condizione principale è che le classi e oggetti del sistema riflettano il dominio:
 - Le entità del mondo reale hanno una coerenza intrinseca
 - Se le classi del sistema riflettono il modello, saranno anch'esse coerenti



Struttura gerarchica del sistema

- Non si parla delle gerarchie di ereditarietà, ma del tipo **contenitore-contenuto** a vari livelli
- Nella maggior parte dei domini applicativi, si hanno oggetti composti di altri oggetti di livello inferiore, a vari livelli
- Le classi OO devono riflettere questa situazione
- Ad esempio:
 - Una società è composta da divisioni
 - Una divisione è composta da dipartimenti
 - Un dipartimento contiene: impiegati, macchinari, progetti, ecc.
 - Ecc..



Struttura gerarchica del sistema

- Le classi del codice devono riflettere questa gerarchia, con oggetti complessi che contengono (associazione, aggregazione, composizione) oggetti di livello inferiore, a vari livelli di annidamento
- Le classi di alto livello eseguono operazioni ad alto livello, “passandone” l'esecuzione alle classi contenute, di livello inferiore
- Ogni classe esegue le operazioni fornite dai propri dati (oggetti contenuti), sino a classi di basso livello che contengono dati elementari (int, float, String, ecc.)



Esempio: Simulatore

- Sia la classe `Simulator`, che contiene la lista degli eventi, ordinati in ordine di tempo e priorità:

```
class Simulator:  
    def __init__(self):  
        self.eventQueue = OrderedQueue()
```

- Il metodo principale è `runSimulation()`, di alto livello:



Esempio: metodo runSimulation()

```
def runSimulation(self):  
    self.setUpSimulationEnvironment()  
    self.initializeQueue()  
    result = self.mainLoop()  
    self.processResult(result)  
    self.tearDownEnvironment()
```



Esempio: metodo `runSimulation()`

- Il metodo `runSimulation()` contiene solo chiamate a metodi livello inferiore
- Ogni metodo ha un nome auto-esplicativo ed esegue operazioni di livello più basso
- Ad esempio, vediamo il metodo `mainLoop()`
- Tale metodo rappresenta ovviamente il ciclo principale della simulazione
- Gli eventi sono estratti dalla coda ed eseguiti



Esempio: metodo mainLoop()

```
def mainLoop(self):  
    while not self.eventQueue.is_empty():  
        currentEvent = self.eventQueue.remove_first()  
        if currentEvent.isEndOfSimulation():  
            return SIMULATION_STOPPED  
        self.setCurrentTime(currentEvent.getTime())  
        currentEvent.execute()  
    return NO_MORE_EVENTS
```



Esempio: metodo `mainLoop()`

- Il metodo `mainLoop()`, oltre a implementare il ciclo, chiama metodi livello inferiore, due dei quali appartengono alla classe `Event` o a sue sottoclassi
- Il metodo chiave è `execute()`, che deve essere implementato in tutte le sottoclassi di `Event` perché ogni evento ha una sua specifica esecuzione
- Entro `Event` e le sue sottoclassi, `execute()` è un metodo di alto livello, che chiamerà al suo interno metodi di livello più basso
- I metodi mostrati sono auto-documentanti: non contengono commenti, ma i commenti qui non sono necessari!



Testing

- ◆ *Scrivete i test prima del codice*
- ◆ Tutto il codice deve avere *Test Unitari*, e superarli sempre al 100%
- ◆ Quando si trova un bug, si scrivono test
- ◆ I *Test di Accettazione* sono eseguiti spesso e se ne pubblicano i risultati
- ◆ Tutti i test devono essere automatici
- ◆ Già visto nella parte di Testing Agile del corso



I ruoli dell'XP

- ◆ **Cliente** (in Scrum, **Product Owner**)
 - Scrive le storie e specifica i test di accettazione. Stabilisce le priorità, spiega le storie, è presente alle sessioni CRC
- ◆ **Tracker** (in Scrum, non presente: sono compiti dello **Scrum Master**)
 - Tiene traccia del progresso del progetto. Una volta o due la settimana gira tra i programmatori, fa domande, intraprende azioni correttive se le cose sembrano andare male



I ruoli dell'XP

- ◆ **Programmatore** (in Scrum, è collettivamente chiamato: **Team**)
 - Stima le storie, definisce i task dalle storie, stima quanto tempo occorre per implementare storie e task, scrive il codice e i test unitari
- ◆ **Coach** (in Scrum, **Scrum Master**)
 - Controlla tutto, si assicura che il progetto resti XP. Aiuta in caso di bisogno



I ruoli dell'XP

- ◆ **Tester** (in Scrum, non presente come ruolo esplicito; è un membro del **Team**)
 - Implementa ed esegue i test di accettazione. Rappresenta graficamente i risultati, quando i risultati regrediscono si assicura che il gruppo lo sappia
- ◆ **Manager** (in Scrum, non presente: è esterno ai ruoli di Scrum)
 - Convoca le riunioni, si assicura che siano convocate le riunioni giuste, tiene traccia dei risultati delle riunioni, li passa al Tracker, isola il gruppo dal management superiore



Il progetto C3 (Chrysler Comprehensive Compensation)

- ◆ Scopo: rimpiazzare le applicazioni utilizzate per il calcolo delle paghe e stipendi mensili in Chrysler
- ◆ 10.000 stipendi mensili, per lo più di dirigenti e consulenti, quindi *molto vari e complessi*
- ◆ L'ambiente di sviluppo: VisualWorks Smalltalk su una workstation Unix
- ◆ Iniziato nel 1994, è stato riazzerato nel 1997 dopo l'assunzione di Kent Beck come consulente
- ◆ Svolto da una decina di programmatori Chrysler, più tre o quattro consulenti (tra cui Beck e Fowler)



Il progetto C3 (Chrysler Comprehensive Compensation)

- ◆ Il sistema realizzato comprendeva circa 1400 classi, con 20.000 metodi
- ◆ Ad esse vanno aggiunte circa 1300 classi per il test, e circa 400 test funzionali
- ◆ Tutti i test unitari “girano” in 5-10 minuti
- ◆ Si tratta del primo e più clamoroso successo di programmazione XP in Smalltalk
- ◆ Ha innescato l'interesse per l'XP



I cicli dell'XP



Planning/Feedback Loops

